DeployGuard Railway-Like Two-Lane PaaS Architecture

Complete deployment, failure containment, rollback, reconciliation, deletion, UI, and release-readiness specification

Version 3.0 | July 2026

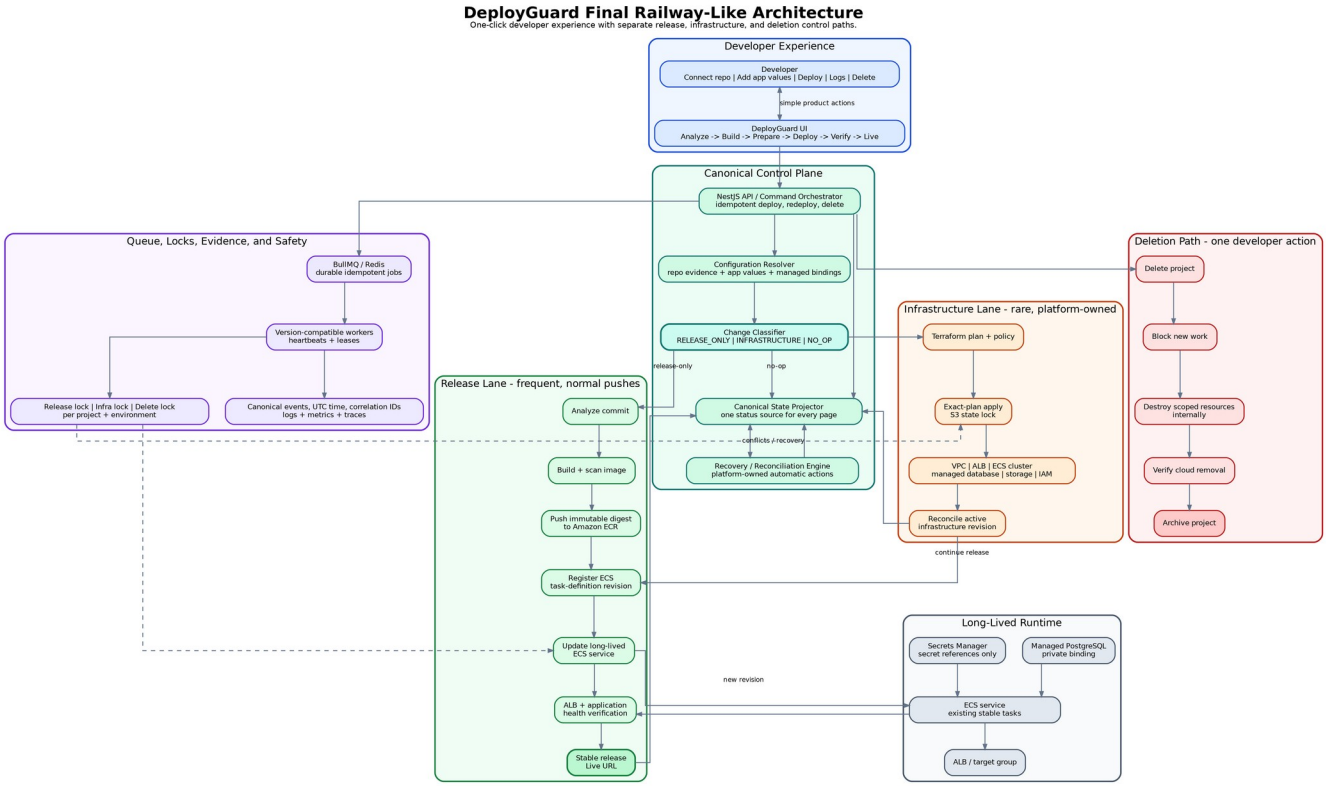


Figure 1. Final target architecture.

Reliability boundary

This document is the final target architecture and implementation specification. It is not evidence that the current DeployGuard code already satisfies the design. Operational reliability is proven only after the acceptance and real-system tests in this report pass.

1. Document Status and Scope

Status
Final architecture and implementation specification. This report supersedes the earlier Railway-like and two-lane reports.

What this report completes

- Two-lane release and infrastructure architecture.
- Semantic Change Classifier with fail-closed behavior.
- Separate release, infrastructure, and deletion state machines.
- Full failure ownership, containment, rollback, and reconciliation behavior.
- Concurrency, idempotency, stale-worker, and workspace-isolation rules.
- Canonical state, event, timestamp, progress, and UI contracts.
- One-click deletion and step-up authentication behavior.
- Migration sequence, test strategy, release gates, and known reliability limits.

What this report does not claim

- It does not guarantee that software can never fail.
- It does not claim the current repository is already synchronized or production-ready.
- It does not replace controlled integration and AWS verification.
- It does not authorize automatic destructive AWS operations outside a verified project scope.

2. Executive Summary

DeployGuard must behave like Railway for a developer with no cloud or DevOps experience: connect a repository, supply unresolved application-owned values, click Deploy, and receive a live URL. Cloud provisioning, release orchestration, recovery, rollback, Terraform state, ECS, networking, secrets, and deletion remain platform-owned.

Primary architecture decision
Ordinary code pushes use the release lane and do not run Terraform. Terraform is reserved for first deployment and genuine infrastructure changes. A separate durable deletion path removes project resources without exposing cloud operations to the developer.

Reliability objective

The goal is not “no errors ever.” The goal is that failures are contained at the smallest safe boundary, the last stable release is protected, platform-owned failures do not become developer tasks, and every UI page reports the same evidence-based state.

3. Product Contract for a Zero-DevOps Developer



Developer may do	DeployGuard must do	Developer must never be asked to do
Connect repository and choose branch	Analyze code and determine runtime requirements	Generate or approve Terraform plans
Provide application-owned secrets	Create and bind managed service credentials	Refresh or repair Terraform state
Click Deploy or Redeploy	Build, provision when needed, release, verify, recover	Inspect ECS task definitions, ARNs, locks, EFS, or Cloud Map
Read application logs	Classify error ownership and preserve stable service	Clean residue or choose a recovery stage
Delete project once	Destroy scoped cloud resources and verify removal	Navigate a multi-page cleanup workflow

Non-negotiable UX rule
A developer-owned error may request an application change. A platform-owned or cloud-owned error may report status, but it must not provide cloud or DevOps instructions to the developer.

4. Final Logical Architecture

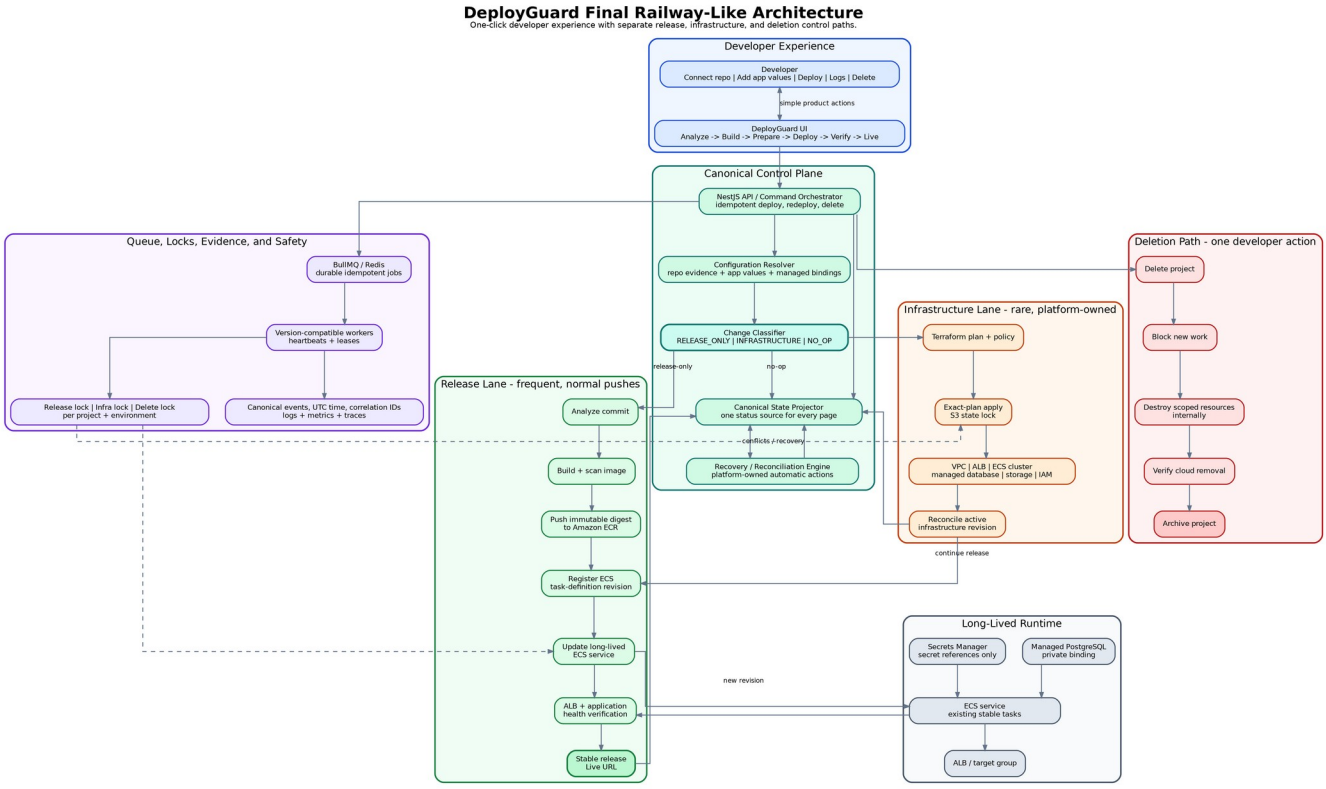


Figure 2. Control plane, two deployment lanes, long-lived runtime, and deletion path.

Component	Responsibility	Required invariant
API / Command Orchestrator	Accept deploy, redeploy, configuration, and delete commands	Duplicate requests return the existing operation
Configuration Resolver	Resolve repository evidence, app-owned values, managed bindings, and defaults	Same semantic input produces identical normalized output
Change Classifier	Route change to release, infrastructure, no-op, or unclassified	Developer never selects a lane
Release Controller	Build and promote task-definition revisions	No Terraform for ordinary pushes
Infrastructure Controller	Plan/apply and reconcile cloud topology	One exact infrastructure mutation per project environment

Component	Responsibility	Required invariant
Deletion Controller	Block work, destroy, verify, and archive	One durable deletion operation
Canonical State Projector	Combine intended state, operations, cloud evidence, and health	Every page consumes the same status
Recovery / Reconciler	Resolve uncertainty, partial operations, and stale evidence	Never auto-destroy after partial failure
Versioned Workers	Execute durable BullMQ jobs	Stale or incompatible workers cannot process jobs

5. Semantic Change Classifier

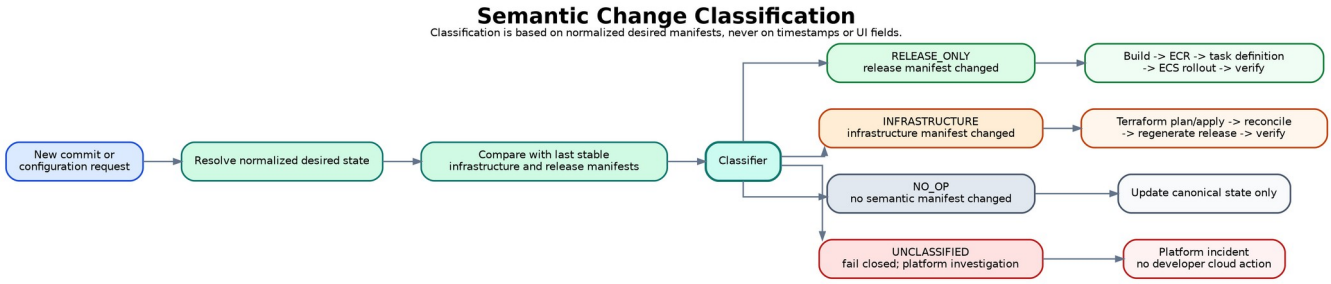


Figure 3. Change classification based on normalized manifests.

Classifier algorithm

- Resolve a normalized desired infrastructure manifest and desired release manifest.
- Compare each manifest with the last stable revision using deterministic canonical serialization.
- If the infrastructure manifest changes, select INFRASTRUCTURE even when the release manifest also changes.
- If only the release manifest changes, select RELEASE_ONLY.
- If neither semantic manifest changes, select NO_OP.
- If required evidence is missing or classification is ambiguous, select UNCLASSIFIED and fail closed as a platform incident.

Example	Classification	Why
Source code or image digest	Release-only	Application artifact changed
Existing environment or secret value	Release-only	Runtime revision changed; resource identity did not
Command, CPU, memory, health path with no structural ALB change	Release-only	Task-definition or rollout behavior changed
First deployment	Infrastructure	Runtime resources do not exist
Add/remove PostgreSQL, Redis, storage, domain/TLS, network, IAM structure	Infrastructure	Cloud topology or managed-service identity changed
Project name, description, notification preference	No-op	No deployment manifest changed
Timestamp, audit event, health observation, inventory refresh	No-op	Operational evidence only
Cannot prove which manifest changed	Unclassified	Fail closed; do not guess or ask developer to run Terraform

6. Revision, Snapshot, and Manifest Model

Artifact	Meaning	Mutable?	Relationship
Desired project state	Current developer intent	Yes, through controlled commands	Source for new immutable operations

Artifact	Meaning	Mutable?	Relationship
Infrastructure revision	Normalized cloud topology and managed-service identity	Immutable after creation	Active revision used by releases
Release revision	Image, command, runtime values, secret references, task resources	Immutable after creation	References exact infrastructure revision
Operation snapshot	Frozen input for one build, infra mutation, or deletion	Immutable	Prevents live settings from changing an in-flight operation
Terraform input manifest	Exact infrastructure inputs	Immutable per infra operation	Produces plan fingerprint
Plan artifact	Exact saved tfplan	Immutable and rehashed	Approved/applied once
Stable release marker	Last verified healthy release	Mutable pointer, append-only history	Rollback target

Required fingerprints

- infrastructureManifestFingerprint
- releaseManifestFingerprint
- buildFingerprint and immutable image digest
- taskDefinitionFingerprint
- terraformInputFingerprint
- terraformPlanFingerprint and SHA-256 artifact hash
- operationSnapshotFingerprint

Determinism rule

GeneratedAt, updatedAt, event timestamps, progress, health status, inventory status, cleanup status, row ordering, and map iteration order must never influence semantic fingerprints.

7. Release Lane - Normal Pushes



Release sequence

- Create one immutable release operation for the commit and active infrastructure revision.
- Resolve application requirements and reject only unresolved developer-owned inputs.
- Build and scan the image; push an immutable ECR digest.
- Render a task definition from the release manifest and active infrastructure bindings.
- Acquire the release lock only for promotion, not for the entire build.
- Register the task-definition revision and update the existing ECS service.
- Verify ECS rollout, target health, application health, and dependency readiness.
- Promote the candidate to stable only after verification; otherwise preserve or restore the previous stable revision.

Release-lane invariants

- Terraform is never invoked for a release-only change.
- Multiple builds may run concurrently; only one candidate may promote per project environment.
- A superseded candidate cannot update ECS after a newer candidate is selected.
- A failed build or scan never changes the running service.
- A failed rollout or health check triggers rollback or preserves the prior stable tasks.
- A release references the exact active infrastructure revision and revalidates it before promotion.

8. Infrastructure Lane - Rare Changes



Infrastructure sequence

- Create an immutable infrastructure operation and run-isolated Terraform workspace.
- Acquire the infrastructure lock and verify no deletion operation owns the project.
- Generate deterministic Terraform inputs and a saved plan.
- Validate plan policy, resource scope, plan hash, and expected replacements or destruction.
- Apply the exact plan once under remote-state locking and durable apply claim.
- Reconcile Terraform state, observed AWS resources, managed-service readiness, and active infrastructure revision.
- Regenerate the newest valid release manifest against the new infrastructure revision and continue through the release lane.

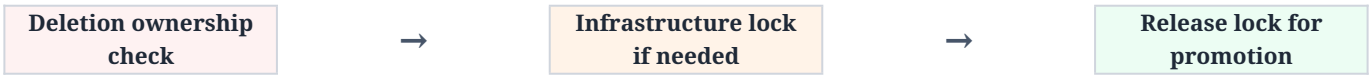
When Terraform runs

Runs Terraform	Does not run Terraform
First project deployment	Normal source-code push
Add/remove managed database, Redis, storage, domain/TLS, network, or IAM structure	New image digest
Infrastructure capacity or persistent backup policy changes	Existing secret value rotation
Region or public/private exposure changes	Task command or app runtime values
Project deletion	Display metadata, health observations, audit records

9. Concurrency, Locking, and Workspace Isolation

Lock	Scope	Protects	Priority
Deletion lock	projectId + environment	Entire deletion lifecycle and block-new-work state	Highest
Infrastructure lock	projectId + environment	Terraform plan/apply, managed-service mutation, reconciliation	Higher than release promotion
Release lock	projectId + environment	ECS promotion, rollout, health, stable marker	Normal
Build idempotency	releaseId / commit	Build, scan, ECR digest production	Parallel across candidates

Lock ordering



- No code path may acquire locks in reverse order.
- A release waits while an infrastructure apply is active, then regenerates against the reconciled infrastructure revision.
- A newer release may build while an older release promotes, but cannot promote until the release lock is available.
- Deletion blocks creation of new release and infrastructure operations before destroying resources.

Workspace isolation

/workspaces/<projectId>/<environment>/<operationId>/ must contain the run-specific Terraform directory, TF_DATA_DIR, backend configuration, variable file, plan file, rendered task definition, and temporary evidence. No two operations may share a local plan path or generated input file.

10. Authoritative State Machines

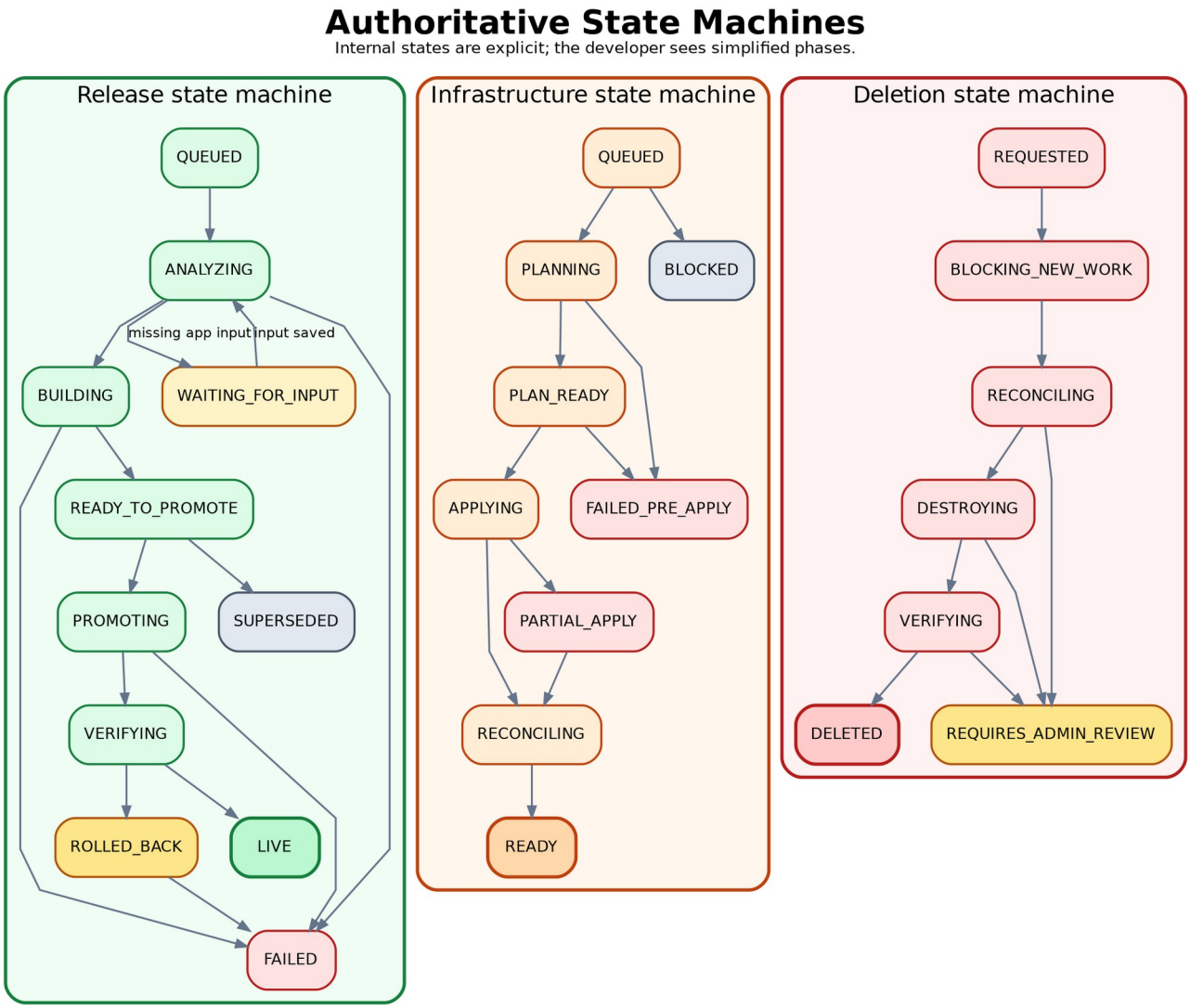


Figure 4. Separate state machines for release, infrastructure, and deletion.

State-transition rules

- A state changes only through a validated transition command.
- A stage or operation becomes successful only after an explicit completion event.
- Queued, lock-acquired, approval-recorded, or heartbeat events are never success evidence.
- Terminal failure overrides older waiting or approval evidence.
- The current stable release is separate from the latest attempted release.
- A partial infrastructure apply enters reconciliation instead of failed-cleanup or automatic destroy.
- A deletion operation remains authoritative until DELETED or REQUIRES_ADMIN_REVIEW.

11. Failure Ownership and Developer Messaging

Owner	Examples	Developer message	Platform action
Developer	Missing app value, build error, app not listening, application health logic	Explain the application-side correction and show relevant logs	Stop at release boundary; preserve stable deployment
Platform	Fingerprint mismatch, stale worker, queue duplication, state projection conflict, invalid AWS CLI usage	DeployGuard encountered an internal issue. Existing resources are preserved.	Create incident evidence, reconcile, retry internally when safe
Cloud provider	Quota, throttling, temporary regional/service failure	Cloud provider is temporarily blocking deployment; DeployGuard will retry.	Backoff, retry, or request admin action for hard quota
Unknown	Evidence contradicts or is insufficient	DeployGuard is verifying deployment state.	Freeze risky actions and reconcile before deciding

Developer messaging rule

Never show Terraform plan generation, state refresh, lock repair, residue cleanup, ECS JSON, ARN repair, or stage-number recovery as a developer action.

12. Complete Failure Containment and Recovery Model

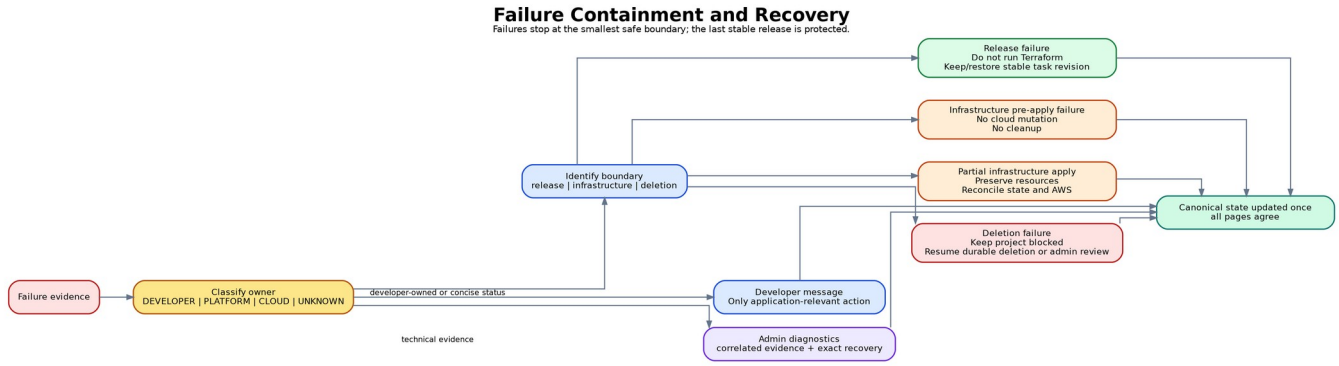


Figure 5. Failure ownership, containment, and canonical-state update.

Previous versus final behavior

Failure	Previous combined pipeline	Final architecture
Build or scan failure	Could be mixed with Terraform and recovery state	Release fails before promotion; infrastructure and stable release unchanged
Plan failure or stale plan	Developer repeatedly generated/approved plans	Infrastructure controller stops or regenerates internally; developer receives no Terraform task
Partial apply	State and UI could disagree or suggest cleanup	Infrastructure enters reconciliation; valid resources are preserved
ECS rollout failure	Could be confused with infrastructure failure	Release controller restores or retains prior stable task revision
Health failure	Could trigger Terraform/recovery loop	Release fails verification; infrastructure remains; previous stable release stays live
Two pushes	Runs could collide on state and plan files	Builds may overlap; newest valid release promotes under release lock
Worker restart	Old code could consume new jobs	Worker compatibility gate and durable idempotency prevent stale execution
Deletion failure	Multi-page cleanup and uncertain outcome	Durable deletion remains blocked/in-progress and resumes or enters admin review

Stage-by-stage failure matrix

Boundary	Failure result	Cloud impact	Automatic recovery	Developer-facing action
Repository access / analysis	Release FAILED or WAITING_FOR_INPUT	None	Retry repository read; request only missing app-owned input	Fix repository access, branch, or app value
Build / dependency install	Release FAILED	None; existing service unchanged	No automatic source-code fix; retain evidence	Review build logs
Security scan / release policy	Candidate rejected	None; image not promoted	Retention cleanup only	Fix vulnerable or blocked dependency when policy requires
Classifier unclassified	Platform incident	None	Freeze deployment and diagnose manifest evidence	No cloud action
Terraform plan	Infrastructure FAILED_PRE_APPLY	None	Regenerate internally only after deterministic cause is resolved	No Terraform action
Plan expires or desired state changes	Old infra operation superseded	None	Create new immutable infra operation internally	No plan-generation action
Terraform apply before mutation	Infrastructure FAILED_PRE_APPLY	None	Retry only when failure is transient and exact plan remains valid	No cleanup
Terraform partial apply	Infrastructure PARTIAL_APPLY	Some resources may exist	Read state + AWS, create reconciliation plan, preserve valid resources	No destroy or state command
Managed service readiness	Infrastructure RECONCILING / BLOCKED	Provisioned infrastructure preserved	Retry service readiness and binding reconciliation	No cloud action
Task-definition registration	Release FAILED	Infrastructure unchanged	Correct platform rendering if platform-owned; otherwise fail candidate	Review app configuration only when developer-owned
ECS service rollout	Release FAILED / rollback	Previous tasks preserved or restored	ECS circuit breaker or explicit stable revision rollback	Review app logs when code-owned
ALB / application health	Release ROLLED_BACK or FAILED	Infrastructure preserved	Collect task, target, and application evidence; rollback	Fix app port/health/dependency only when proven
Stable marker persistence	Release VERIFYING until reconciled	Healthy service remains	Reconcile runtime evidence and persist marker idempotently	No action
Worker crash / duplicate delivery	Operation resumes from durable claim	No duplicate apply/promotion	Lease expiry, idempotency check, reconcile external side effect	No action
Deletion destroy failure	Deletion REQUIRES_ADMIN_REVIEW or retries	Project remains blocked; partial removal possible	Reconcile exact state and continue; never reopen deployments automatically	No Terraform cleanup action

13. Release Rollback and Stable Promotion

Promotion rule

A candidate becomes stable only after the ECS rollout is complete, target health is healthy, the application health endpoint succeeds, required managed services are ready, and the release still references the active infrastructure revision.

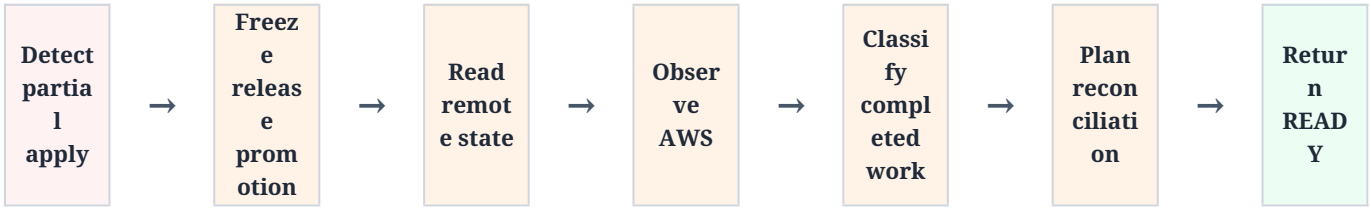
Rollback rule

- Keep the prior stable task-definition revision and release manifest until the new candidate is promoted.
- Use ECS deployment circuit breaker where appropriate, but also persist DeployGuard-owned rollback evidence.
- If rollout or health verification fails, update the service back to the prior stable revision when required.

- Verify the rollback itself before reporting the previous version as available.
- Do not invoke Terraform for a release rollback.
- Do not recommend infrastructure cleanup for a release failure.

Condition	Result
No previous stable release exists	Candidate failure leaves project not live; infrastructure remains for diagnosis
Previous stable release is healthy	Restore or preserve previous revision and mark candidate failed
Previous stable release also fails current verification	Enter runtime reconciliation; do not falsely claim rollback success
Infrastructure revision changed during candidate build	Regenerate candidate runtime manifest before promotion

14. Partial Terraform Apply Reconciliation



- Never automatically destroy after a partial apply.
- Preserve remote state, plan metadata, apply operation identity, and sanitized Terraform output.
- Compare the intended infrastructure revision with Terraform state serial and observed AWS resources.
- Import is not an automatic default; use only when ownership is proven and policy explicitly permits it.
- Generate a new internal reconciliation plan against current state and review destructive actions through platform policy.
- Block release promotion until the active infrastructure revision is READY.
- Expose detailed evidence only in admin diagnostics.

15. Canonical State, Events, Progress, and Time

Canonical state response

Field	Purpose
projectStatus	Live, deploying, failed, deleting, deleted, verifying, or blocked
activeReleaseId / stableReleaseId	Separate latest attempt from stable service
activeInfrastructureRevision	Infrastructure revision used by releases
operationType and operationId	Release, infrastructure, or deletion identity
phase and progress	Five developer-facing phases with monotonic progress
errorOwner and failureClass	Developer, platform, cloud, or unknown classification
userActionRequired	True only for a concrete developer-owned action
cleanupRequired	False for pre-apply and release failures; derived only from verified deletion/reconciliation evidence
lastVerifiedAt and evidenceFreshness	Prevent confident status from stale evidence

Canonical event schema

- eventId, projectId, environment, operationId, operationType, phase, internalStage, attempt, eventType, sequenceNumber
- occurredAtUtc assigned by an authoritative clock, receivedAtUtc, startedAtUtc, finishedAtUtc, durationMs
- workerId, workerVersion, correlationId, sanitized reasonCode, errorOwner, evidence references

Progress model



- Progress never moves backward.
- Internal revalidation remains inside the current high-level phase.
- Queued is not running; lock acquisition is not completion.
- The frontend never calculates duration or status independently.
- All timestamps are stored in UTC and converted only for display.

16. Worker, Queue, and Idempotency Safety

Control	Required behavior
Worker compatibility	Worker registers app version, git SHA, schema version, start time, and heartbeat; incompatible workers cannot claim jobs
Job identity	Stable idempotency key per build, promotion, infrastructure revision, and deletion request
Durable claim	External side effect requires a durable claim before execution and reconciliation after uncertain completion
Duplicate delivery	Second delivery returns or resumes existing operation; executor runs at most once
Lease and heartbeat	Expired lease allows controlled recovery only after checking external side effects
Dead-letter handling	Repeated platform failure creates admin incident; it does not create endless new developer-visible runs
Queue cleanup	Terminal operations are retained for audit and compacted by policy, not silently reused

17. Security and Secret Handling

- Application-owned secrets and managed credentials are stored in Secrets Manager and referenced by ECS.
- DB_PASSWORD and JWT_SECRET must never appear in ordinary environment arrays, logs, events, API responses, plan summaries, or activity details.
- DB_HOST, DB_PORT, DB_NAME, and approved non-sensitive values may appear as environment entries.
- DATABASE_URL is included only when current repository evidence proves the application uses it.
- Managed-service values cannot be overridden by historical user variables.
- No key may exist in both ECS environment and secrets.
- Every plan/apply/deletion action is authorized by project scope and environment ownership.
- Admin diagnostics expose allowlisted metadata and sanitized reason codes only.

18. One-Click Deletion and Authentication



Developer experience

- One confirmation modal explains that application and managed database data will be removed.
- The delete button disables immediately and returns a durable deletion operation ID.
- The project page becomes a persistent deletion progress screen that survives refresh.

- No Terraform, state, residue, or resource-selection page is shown.
- Completion is shown only after cloud and state verification.

Backend deletion sequence

- Acquire deletion ownership and block all new release/infrastructure operations.
- Cancel queued candidates and reconcile any operation already mutating AWS.
- Back up state metadata and generate a project-scoped internal destroy operation.
- Handle backup-vault, deletion-protection, secret recovery windows, and dependency ordering internally.
- Verify remote state and observed AWS resources; classify inactive/pending-deletion records correctly.
- Archive the project and retain read-only audit history.

Authentication

A valid recent session should be sufficient. When security policy requires step-up authentication, the user authenticates once and returns directly to the same pending deletion action using a short-lived, single-use action token. An authentication loop is a platform defect.

19. Developer and Admin UI Specification

Developer pages

Page	Shows	Must not show
Overview	Live URL, current phase, latest and stable release, Deploy/Redeploy, Delete	Terraform, resource inventory, state, locks, residue
Deployments	Simple history, five phases, application-relevant result	20 internal stages as primary workflow
Environment	App-owned variables and read-only managed binding summary	Managed passwords or cloud identifiers
Logs	Build and application logs with concise platform messages	Raw secrets or unrelated cloud diagnostics
Metrics	Availability, latency, CPU, memory, release health	Terraform metrics
Settings	Repository, branch, app settings, domain, deletion	Manual cloud cleanup

Admin-only diagnostics

- Infrastructure and release manifests and fingerprints.
- Terraform plan/apply identity, state serial, lock, and reconciliation evidence.
- Queue jobs, worker versions, heartbeats, leases, duplicate deliveries, and dead letters.
- Observed AWS resources, ownership confidence, and deletion evidence.
- Detailed internal stages, failure classes, and sanitized technical output.
- State-proof endpoint returning SYNCHRONIZED, STALE, CONTRADICTORY, or UNVERIFIED.

20. Backend Command and API Contracts

Command / endpoint concept	Contract
POST deploy	Create or return one idempotent operation for the current desired state; user does not choose lane
POST configuration	Persist developer intent, produce semantic revision change, optionally trigger deploy
GET project state	Return canonical state only; all pages use this response
GET operation events	Return ordered sanitized events for one operation
POST redeploy	Create release candidate from current stable infrastructure without forcing Terraform

Command / endpoint concept	Contract
POST delete	Create or return durable deletion operation; block new work immediately
GET state-proof (admin)	Return revision, operation, worker, queue, Terraform, AWS, and health evidence with conflict classification
POST reconcile (admin/internal)	Run bounded reconciliation; no arbitrary manual state mutation from developer UI

Command invariants

- Every write command has an idempotency key and authorization scope.
- Responses return operation identity and canonical state, not inferred frontend status.
- A command never silently starts both a new operation and resumes an old one.
- A terminal operation cannot return to waiting or approval-required state.
- No public developer endpoint exposes Terraform apply, state refresh, residue cleanup, or stage-selective recovery.

21. Observability and Reconciliation

- Correlate API request, operation, BullMQ job, worker, Terraform action, AWS action, ECS rollout, health check, and UI state using projectId, environment, operationId, attemptId, workerId, and correlationId.
- Measure queue delay, build duration, lock wait, plan duration, apply duration, rollout duration, rollback duration, health verification, deletion duration, and reconciliation conflicts.
- Run a background reconciler comparing intended state, active operations, Terraform state, observed AWS state, ECS status, ALB target health, and application health.
- When evidence is stale or contradictory, show “Checking deployment status” and block destructive or promotional actions.
- Aggregate noisy resource-discovery activity for developers; retain detailed events only for admin audit.

22. Migration from the Current DeployGuard Implementation

Phase	Change	Exit condition
0. Freeze unsafe actions	Disable repeated plan generation, manual approval loops, and cleanup recommendations while state is untrusted	No new contradictory runs
1. Canonical state	Create one project-state projector and admin state-proof endpoint	All pages agree on one operation and evidence freshness
2. Separate domains	Introduce infrastructure/release manifests and revisions; migrate semantic fingerprints	Ordinary metadata writes do not stale deployments
3. Classifier	Route release-only, infrastructure, no-op, unclassified	Normal push demonstrably bypasses Terraform
4. Release lane	Implement release lock, task-definition promotion, health verification, stable rollback	Five consecutive release-only deployments succeed
5. Infrastructure lane	Harden isolated workspaces, exact-plan apply, reconciliation	First deploy and managed-service change succeed once
6. Worker safety	Version gate, heartbeat, lease, idempotency, duplicate delivery recovery	Stale worker cannot consume jobs
7. Deletion	Implement durable one-click delete and step-up auth continuation	Two disposable projects delete successfully
8. UI simplification	Remove Terraform/cloud operations from developer pages	Zero cloud/DevOps instructions in developer flow
9. Operational proof	Run concurrency, restart, failure, rollback, partial apply, and AWS tests	Release gates in Section 24 pass

Migration warning

Do not layer the new architecture on top of conflicting legacy current-state, recovery, and fingerprint systems. Select one authoritative implementation per concept and deprecate old writers/readers deliberately.

23. Test Strategy

Local and integration tests

- Deterministic manifest and fingerprint property tests.
- Classifier tests for release-only, infrastructure, no-op, and unclassified cases.
- Release state-machine transition and rollback tests.
- Infrastructure exact-plan, locking, partial-apply, and reconciliation tests.
- Deletion idempotency, restart, step-up authentication, and partial-removal tests.
- Real PostgreSQL advisory-lock tests with separate connections.
- Real Redis/BullMQ duplicate delivery, lease expiry, and worker restart tests.
- Run-isolated workspace and exact-file identity tests.
- Canonical state and cross-page contract tests.
- UTC event ordering, sequence number, and duration consistency tests.
- Secret destination and redaction tests.

Controlled AWS scenarios

- Fresh first deployment.
- Five ordinary code pushes that never invoke Terraform.
- Two same-project pushes building concurrently.
- New push during an infrastructure apply.
- Managed service addition and removal.
- ECS rollout failure and verified rollback.
- Application health failure with infrastructure preserved.
- Terraform partial apply followed by reconciliation.
- Worker restart before and after external side effects.
- One-click deletion, deletion restart, and cloud verification.

24. Final Acceptance and Release Gates

Gate	Pass condition
Release path	Five consecutive normal pushes: zero Terraform calls, one promotion at most, healthy final release
Concurrency	Two builds overlap safely; newest valid candidate wins; no duplicate ECS promotion
Infrastructure serialization	Only one plan/apply per project environment; release waits and regenerates safely
Worker safety	Stale version rejected; duplicate job invokes external executor at most once
Failure containment	Build, scan, rollout, and health failures preserve existing infrastructure and stable release
Partial apply	No auto-destroy; state/AWS reconciliation reaches READY or bounded admin review
Canonical state	All developer and admin pages agree on operation, phase, owner, progress, and timestamp basis
Developer boundary	No normal user sees Terraform, state, lock, residue, ECS, EFS, or Cloud Map instructions

Gate	Pass condition
Deletion	Two disposable projects delete from one confirmation; state and active billable resources verified absent
Security	No plaintext secret leakage; exact ECS environment/secrets destination rules pass
Operational proof	Real AWS deployment, rollback, managed-service readiness, ALB health, and deletion evidence retained

Required success statement

DeployGuard is operationally proven only after the applicable release gates pass using real PostgreSQL/Redis behavior and controlled AWS operations. A build, unit-test report, mocked plan, or AI completion summary is not sufficient.

25. Known Limits and Non-Negotiable Decisions

Known limits

- External AWS outages, quotas, and eventual-consistency delays cannot be eliminated; they can be classified, retried, and communicated correctly.
- No architecture can promise zero bugs. The design promises bounded failure, evidence, rollback, and safe recovery behavior.
- Some infrastructure changes may require downtime or replacement; the platform must predict and communicate impact before mutation.
- Deletion of KMS keys, secrets, and inactive ECS records may remain visible during provider-defined retention or deletion windows.

Non-negotiable decisions

- Terraform is absent from ordinary release-only pushes.
- The developer never selects a pipeline lane or cloud recovery action.
- One canonical state projector is authoritative for every page.
- One infrastructure mutation and one release promotion are serialized per project environment.
- The previous stable release is protected until the candidate is verified.
- Partial infrastructure failure triggers reconciliation, not automatic destroy.
- Deletion is a single durable product action, not a technical cleanup workflow.
- Current implementation reliability is never inferred from document quality or mocked tests.

26. Final Recommendation and Reliability Verdict

Adopt this report as the final target architecture and implementation specification for DeployGuard. Replace the current combined Terraform-driven deployment model with a release-first two-lane model, explicit state machines, deterministic manifest classification, platform-owned recovery, and one-click deletion.

Verdict

The architecture in this report is complete enough to guide implementation because it now specifies normal flows, state transitions, failures, rollback, partial-apply reconciliation, concurrency, deletion, UI contracts, security, migration, and release proof. It is reliable as a target design. The current codebase remains unproven until the gates in Section 24 pass.

- Normal developer flow: Connect repository -> provide app values -> Deploy -> Live.

- Normal code push: Build -> ECR -> task revision -> ECS rollout -> verify; no Terraform.
- Infrastructure change: Plan/apply once -> reconcile -> release; developer does not operate it.
- Failure: contain at the smallest boundary, preserve stable service, classify owner, recover internally.
- Deletion: confirm once -> durable progress -> verified cloud removal -> archive.

Prepared as the superseding final DeployGuard architecture and recovery specification